

SONARQUBE CLOUD WALKTHROUGH

Objective:

The objective of integrating SonarQube Cloud into the development process is to continuously monitor and improve code quality throughout the software lifecycle. It automatically analyzes source code for potential bugs, security vulnerabilities, code smells, and adherence to coding standards each time changes are committed.

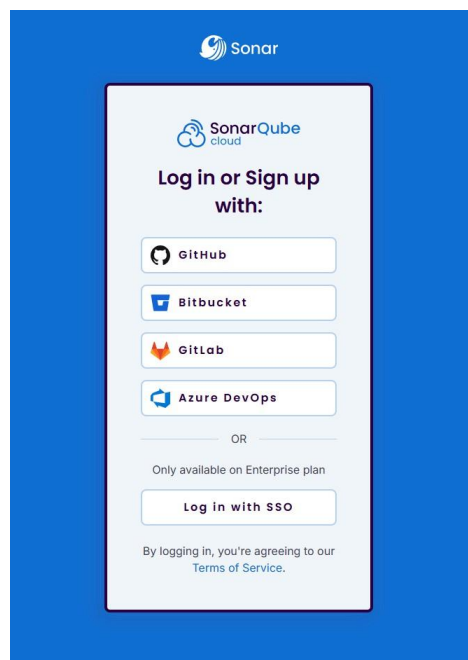
When to Use

- When setting up the CI/CD for the first time.
- Throughout the project lifecycle.
- Optionally, by having the plugin installed on the IDE to detect issues as you code.

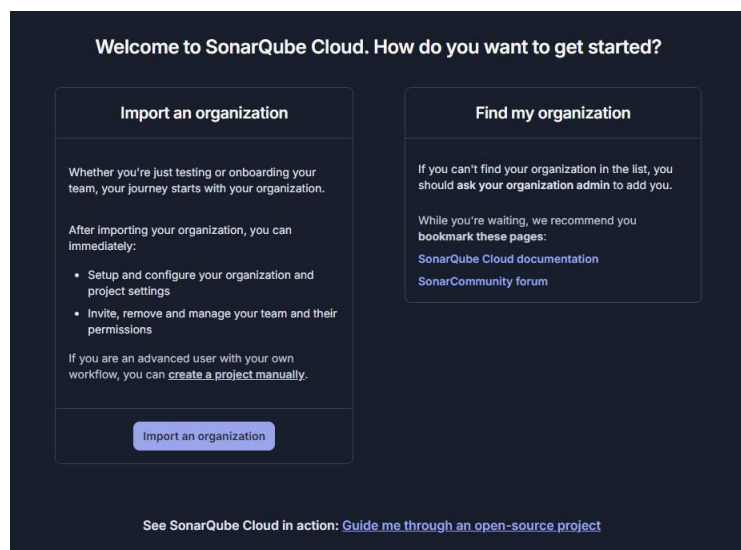
Steps

1. SonarCloud Setup

Login into <https://sonarcloud.io/login> using your GitLab account:

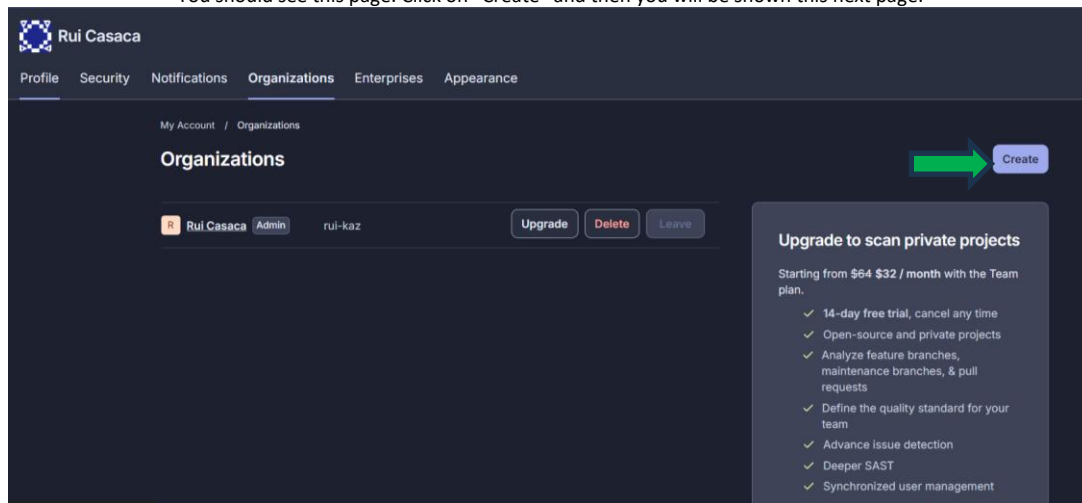


Select "Import an organization":

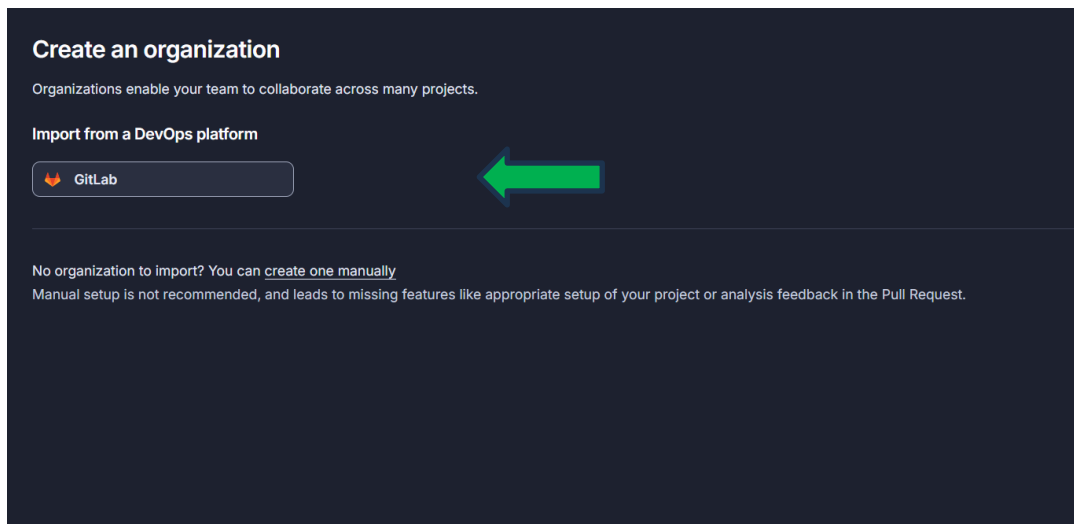


Then, ignore and click on this link, or go to your account on the upper right corner, and select “view all” [Organizations - My Account - SonarQube Cloud](#)

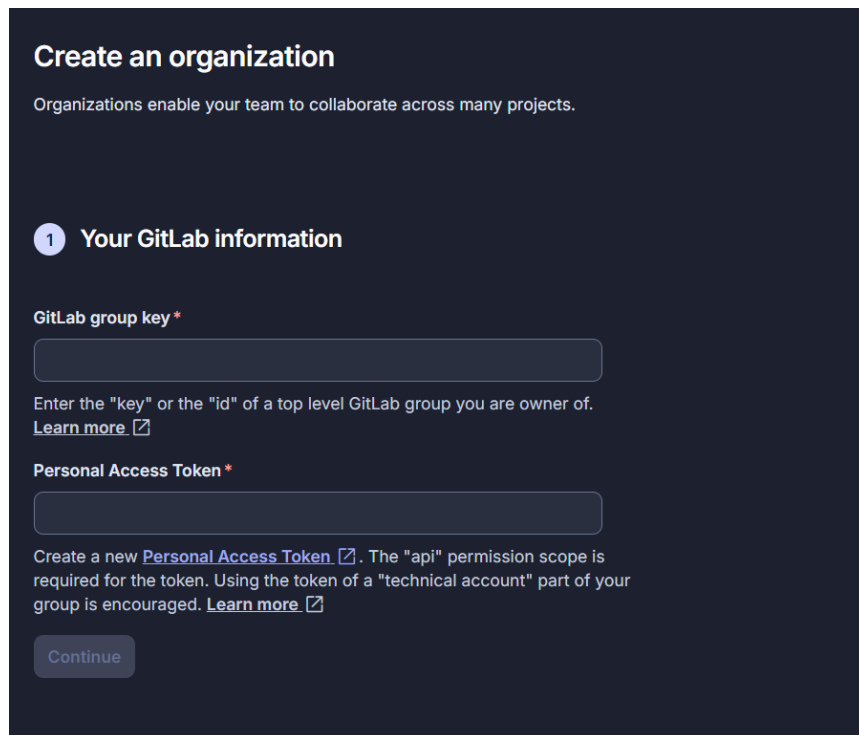
You should see this page. Click on “Create” and then you will be shown this next page:



Select GitLab.



Then it will request a Group Id and personal access token:



Create an organization

Organizations enable your team to collaborate across many projects.

1 Your GitLab information

GitLab group key *

Enter the "key" or the "id" of a top level GitLab group you are owner of. [Learn more](#) 

Personal Access Token *

Create a new [Personal Access Token](#) . The "api" permission scope is required for the token. Using the token of a "technical account" part of your group is encouraged. [Learn more](#) 

[Continue](#)

Click on the "Personal Access Token" link, or just click here: https://gitlab.com/-/user_settings/personal_access_tokens to get your personal access token.

Remember to save your token well since you won't be able to see it after leaving the page.

Click “add new token”, choose only api and crate token.

Personal access tokens



You can generate a personal access token for each application you use that needs access to the GitLab API. You can also use personal access tokens to authenticate against Git over HTTP. They are the only accepted password when you have Two-Factor Authentication (2FA) enabled.

Token name
SonarCloud

Description (optional)
SonarCloud token

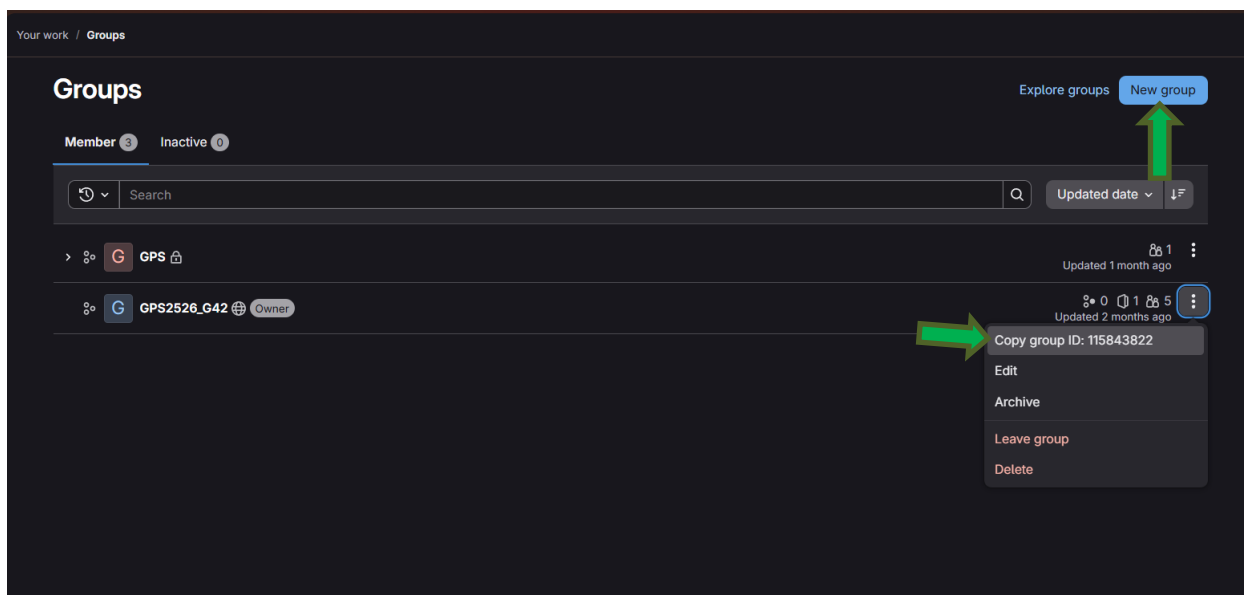
Expiration date
2026-01-01 
An administrator has set the maximum expiration date to 2026-10-30. [Learn more.](#)

Select scopes
Scopes set the permission levels granted to the token. [Learn more.](#)

- ☐ read_user
Grants read-only access to your profile through the /user API endpoint, which includes username, public email, and full name. Also grants access to read-only API endpoints under /users.
- ☐ read_repository
Grants read-only access to repositories on private projects using Git-over-HTTP or the Repository Files API.
- ☐ read_virtual_registry
Grants read-only access to container images through the dependency proxy in private projects and virtual registries.
- ☐ read_registry
Grants read-only access to container registry images on private projects.
- ☐ read_api
Grants read access to the API, including all groups and projects, the container registry, and the package registry.
- ☐ self_rotate
Grants permission for token to rotate itself.
- ☐ write_repository
Grants read-write access to repositories on private projects using Git-over-HTTP (not using the API).
- ☐ write_virtual_registry
Grants read, write, and delete access to container images through the dependency proxy in private projects.
- ☐ write_registry
Grants write access to container registry images on private projects. You need both read and write access to push images.
- ☒ api 
Grants complete read/write access to the API, including all groups and projects, the container registry, the dependency proxy, and the package registry.
- ☐ all_features
Grants access to GitLab Duo related API endpoints.
- ☐ create_runner
Grants create access to the runners.
- ☐ manage_runner
Grants access to manage the runners.
- ☐ k8s_proxy
Grants permission to perform Kubernetes API calls using the agent for Kubernetes.

[Create token](#) [Cancel](#)

Then, for the Group ID, you need to go to this link of your GitLab: <https://gitlab.com/dashboard/groups> and, on the 3 dots of your project, you'll find your Group ID that you'll need.



Insert the created token back into SonarCloud, as well as the Group ID you've copied, and then just click continue:

Create an organization

Organizations enable your team to collaborate across many projects.

1 Your GitLab information

GitLab group key *

Group ID

Enter the "key" or the "id" of a top level GitLab group you are owner of. [Learn more](#)

Personal Access Token *

Create a new [Personal Access Token](#) . The "api" permission scope is required for the token. Using the token of a "technical account" part of your group is encouraged. [Learn more](#)

Continue

These fields should appear automatically:

2 Import organization details

Import 🚀 GPS2526_G42 into a SonarQube Cloud organization

Name *

GPS2526_G42

Up to 255 characters

Key *

gps2526-g42

Use only lowercase letters, numbers, or hyphens. Must start and end with a letter or number. Up to 255 characters.

[> Add additional info](#)

And here, be careful, because you must select the free option, and continue:

The screenshot displays three pricing plans for SonarQube Cloud. The 'Free' plan is highlighted with a red border. The 'Team' plan includes a 'Free 14-day trial' badge. The 'Enterprise' plan is also visible.

Free	Team Free 14-day trial	Enterprise
For individual developers, students and open-source projects	Essential capabilities for small teams and business	Mission critical flexibility, scalability, and performance.
\$0	Starting at \$64 \$32/month - 50% off	To get started Talk to sales
Select free	Start free trial	Contact sales
What's included - Up to 50k private lines of code - Up to 5 members - Unlimited public lines of code	What's included - Up to 1.9M private lines of code - Unlimited members - Unlimited public lines of code	What's included - From 5M private lines of code - Unlimited members - Unlimited public lines of code
Key features ✓ Open-source and private projects ✓ 30 languages supported ✓ Issue detection and SAST ✓ Analyze the main branch & pull	Everything in Free and ✓ 14-day free trial, cancel any time ✓ Open-source and private projects ✓ Analyze feature branches, maintenance branches, & pull requests ✓ Define the quality standard for your	Everything in Team and ✓ Enterprise-Level Hierarchy ✓ Secure SAML single sign-on (SSO) ✓ Management reporting ✓ Centralized enterprise billing ✓ Enterprise-grade security

After you're redirected to your dashboard, click on "Analyze a new project"

The screenshot shows the SonarQube Cloud dashboard for organization 'GPS2526_G42'. The 'No projects here yet' message is displayed, along with a green arrow pointing to the 'Analyze a new project' button.

URL: sonarcloud.io/organizations/gps2526-g42/projects?view=leak

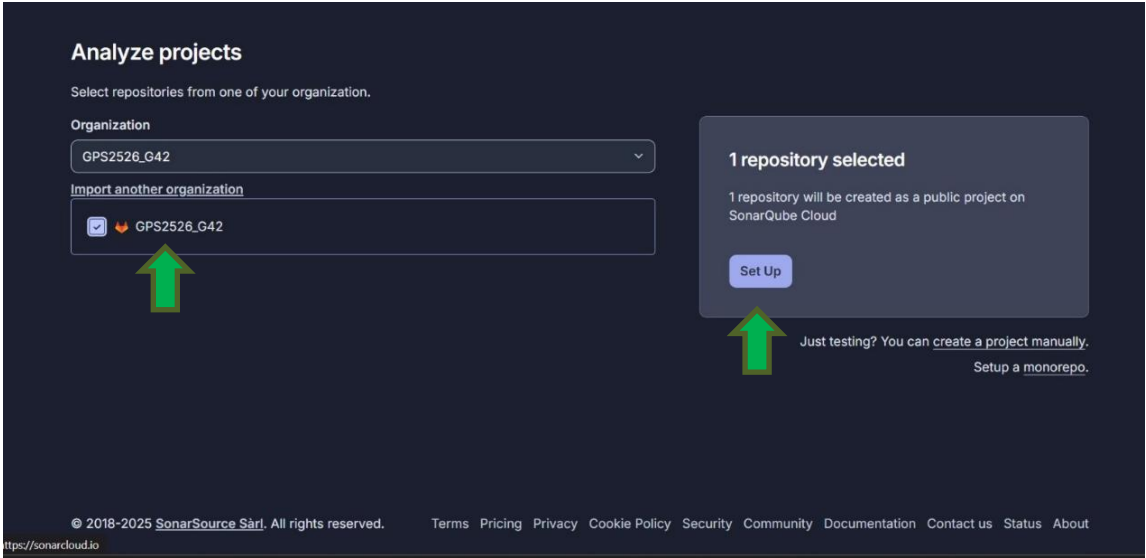
Organization: **GPS2526_G42** [View on GitLab](#) [Free plan](#) Key: gps2526-g42

Navigation: [Projects](#) [Quality profiles](#) [Rules](#) [Quality Gates](#) [Members](#) [Billing and upgrade](#) [Administration](#)

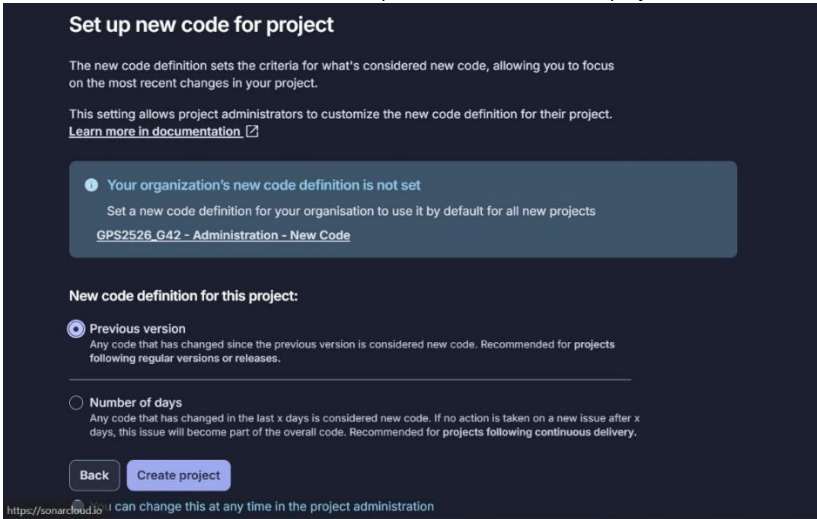
Message: **No projects here yet**
But your organization is all set, you can start analyzing here!

Button: [Analyze a new project](#)

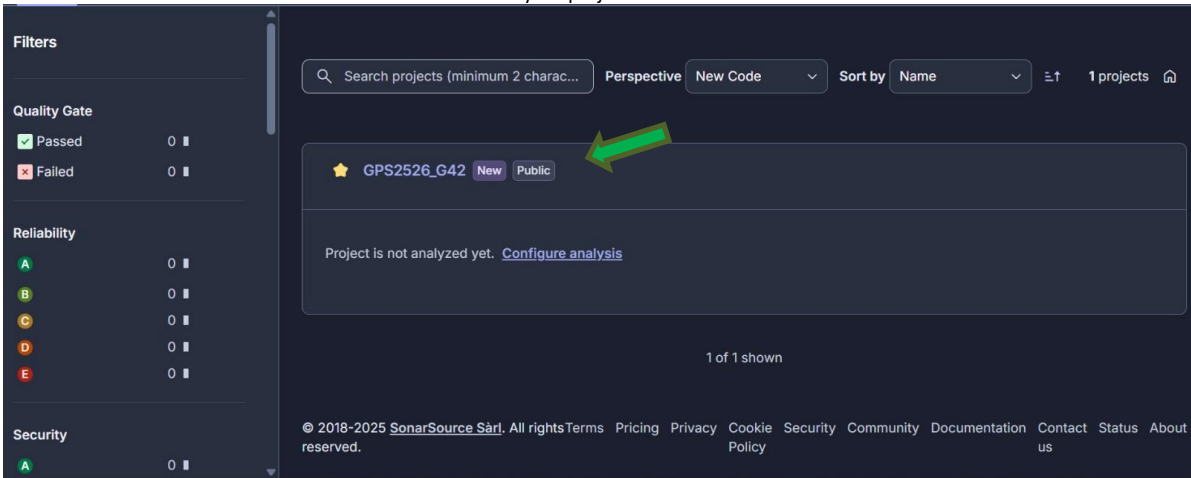
Select your project, and click on “Set Up:



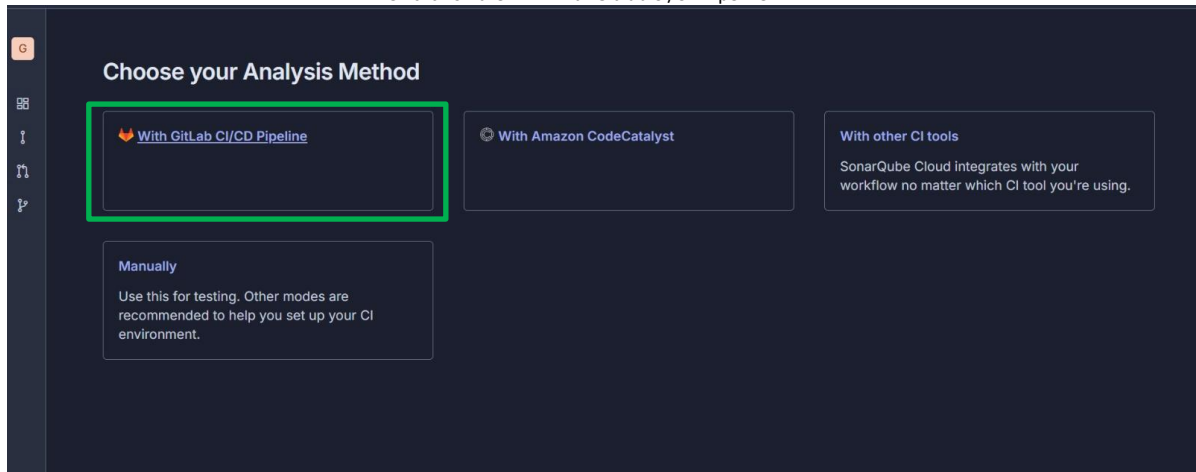
Here maintain the default option and click on “Create project”



Click on your project name:

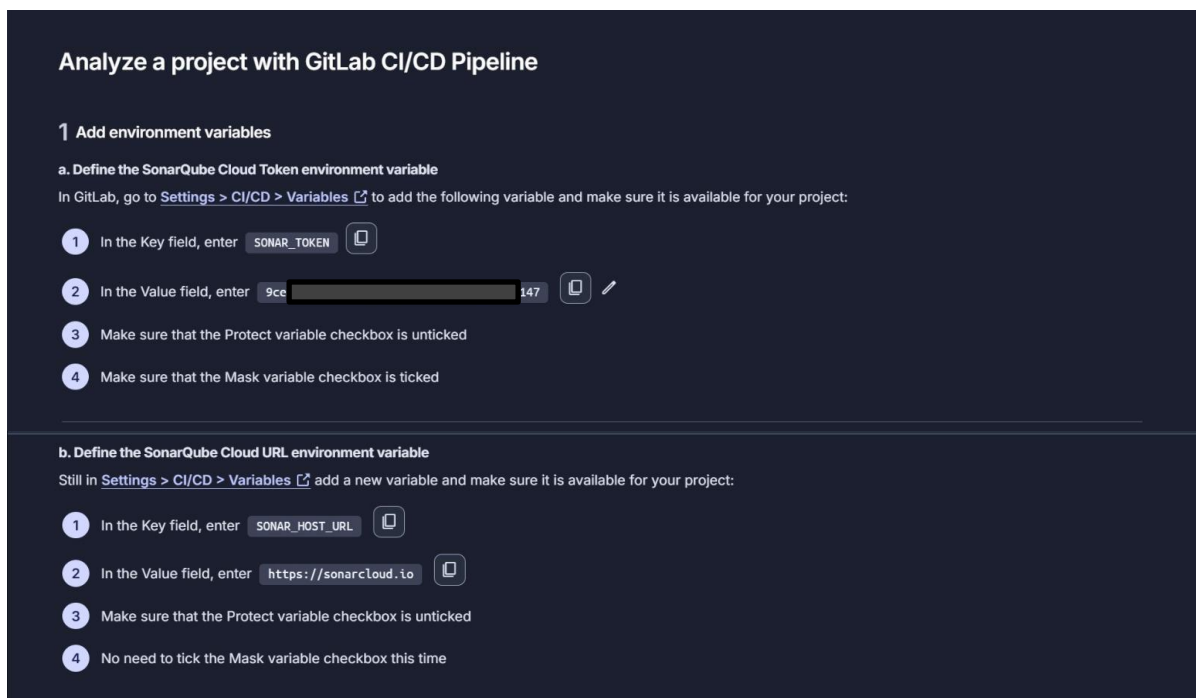


Then click on the link "With GitLab CI/CD Pipeline":



Analyze a Project with GitLab CI/CD Pipeline

Just follow the instructions below. You have to add 2 variables in GitLab.



When you go to the Settings -> CI/CD -> Variables, click in the “Add variable” button.

Minimum role to use pipeline variables
Select the minimum role that is allowed to run a new pipeline with pipeline variables. What are pipeline variables?

- ☐ No one allowed
Pipeline variables cannot be used.
- ☐ Owner
- ☐ Maintainer
- ☒ Developer

[Save changes](#)

Access protected resources in merge request pipelines
Make protected CI/CD variables and runners available in merge request pipelines. Protected resources will only be available in merge request pipelines if both the source and target branches of the merge request are protected. [Learn more.](#)

☒ Allow merge request pipelines to access protected variables and runners

[Save changes](#)

Display manually-defined pipeline variables
Display all manually-defined variables in the pipeline details page after running a pipeline manually. [Learn more.](#)

☐ Display pipeline variables

All manually-defined CI/CD variables and their values are visible to maintainers, which is a security risk if including credentials or other secrets in variables. Do not enable this feature if variables could contain sensitive data. Developers can only view manually-defined variables in their own manual pipelines.

[Save changes](#)

Project variables
Variables can be accidentally exposed in a job log, or maliciously sent to a third party server. The masked variable feature can help reduce the risk of accidentally exposing variable values, but is not a guaranteed method to prevent malicious users from accessing variables. [How can I make my variables more secure?](#)

CI/CD Variables </> 1 [Reveal values](#) [Add variable](#)

Then it will show you this window, and you have to select Masked, and unchecked the “Protect variable”.

Add variable ×

Type
Variable (default) ▾

Environments ⓘ
All (default) ▾

Visibility ⓘ
☐ Visible
Can be seen in job logs.
☒ Masked
Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet [regular expressions requirements](#).
☐ Masked and hidden
Masked in job logs, and can never be revealed in the CI/CD settings after the variable is saved.

Flags
☐ Protect variable
Export variable to pipelines running on protected branches and protected tags only.
☐ Expand variable reference
\$ will be treated as the start of a reference to another variable.

Description (optional)

The description of the variable's value or usage.

Key
SONAR_TOKEN
You can use CI/CD variables with the same name in different places, but the variables might overwrite each other. [What is the order of precedence for variables?](#)

Value
e86
8f
Variable value will be evaluated as raw string.

[Add variable](#) [Cancel](#)

Here you'll insert the key and value from the steps above (Add environment variable). Then click on “Add variable”.

For the second variable, just follow these instructions as the picture shows:

Edit variable [X]

Type
Variable (default) [v]

Environments [?]
All (default) [v]

Visibility [?]
☒ **Visible**
Can be seen in job logs.
☐ **Masked**
Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet [regular expressions requirements](#).
☐ **Masked and hidden**
Masked in job logs, and can never be revealed in the CI/CD settings after the variable is saved.

Flags
☐ **Protect variable**
Export variable to pipelines running on protected branches and protected tags only.
☒ **Expand variable reference**
\$ will be treated as the start of a reference to another variable.

Description (optional)
[Text area]
The description of the variable's value or usage.

Key
[SONAR_HOST_URL]
You can use CI/CD variables with the same name in different places, but the variables might overwrite each other. [What is the order of precedence for variables?](#)

Value
[https://sonarcloud.io]
Variable value will be evaluated as raw string.

[Save changes] [Delete variable] [Cancel]

Here the visibility, select “visible”, and uncheck the “Protect variable”

Finally put the key and value from the steps above here.
Then click “Save changes”.

After these steps, guarantee that your pom.xml file from your project is something like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>pt.isec.gps2526.group42</groupId>
  <artifactId>surpriseme</artifactId>
  <version>0.1.0-SNAPSHOT</version>
  <name>SurpriseMe</name>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>21</maven.compiler.release>
    <javafx.version>21.0.4</javafx.version>
    <junit.jupiter.version>5.10.2</junit.jupiter.version>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <javafx.classifier>win</javafx.classifier>
    <sonar.organization>gps2526-g42</sonar.organization>
```

Read the instructions and update your pom.xml file of your project to have that property (shown above), and create or update a file with the name ".gitlab-ci.yml".

2 Create or update a .gitlab-ci.yml file

What option best describes your project?

JS/TS & Web **Maven** Gradle .NET Python C, C++ or ObjC Flutter or Dart Other (for Go, PHP, ...)

Update your pom.xml file with the following properties:

```
<properties>
  <sonar.organization>gps2526-g42</sonar.organization>
</properties>
```

Copy

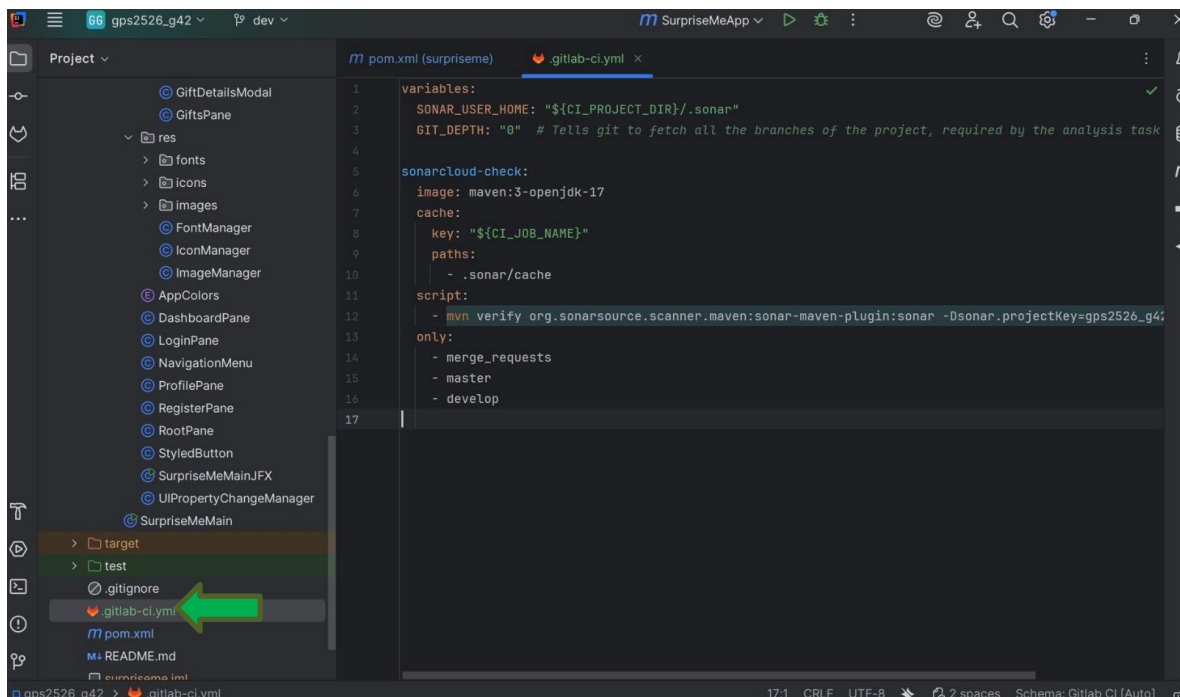
Create or update your .gitlab-ci.yml yml file with the following content:

```
variables:
  SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar" # Defines the location of the analysis task cache
  GIT_DEPTH: "0" # Tells git to fetch all the branches of the project, required by the analysis task
sonarcloud-check:
  image: maven:3-openjdk-17
  cache:
    key: "${CI_JOB_NAME}"
    paths:
      - .sonar/cache
  script:
    - mvn verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -Dsonar.projectKey=gps2526_g42_gps2526_g42
  only:
    - merge_requests
    - master
    - develop
```

Copy

Note that this is a minimal base configuration to just run a SonarQube Cloud analysis on your master branch and merge requests. If you already have a pipeline configured and running you might want to just add this new step to your existing yml file.

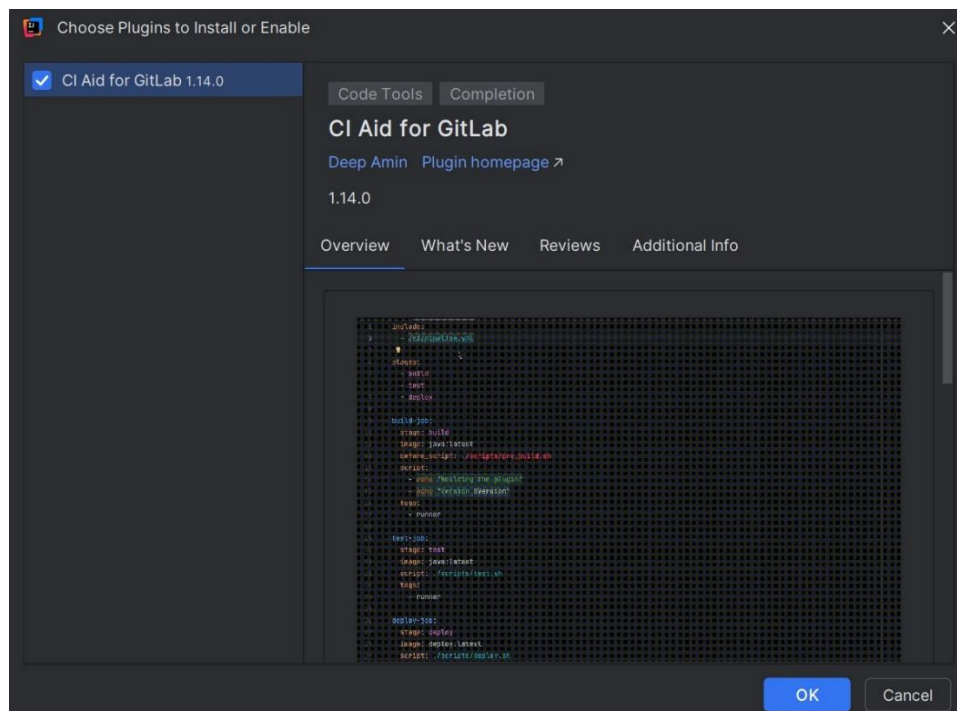
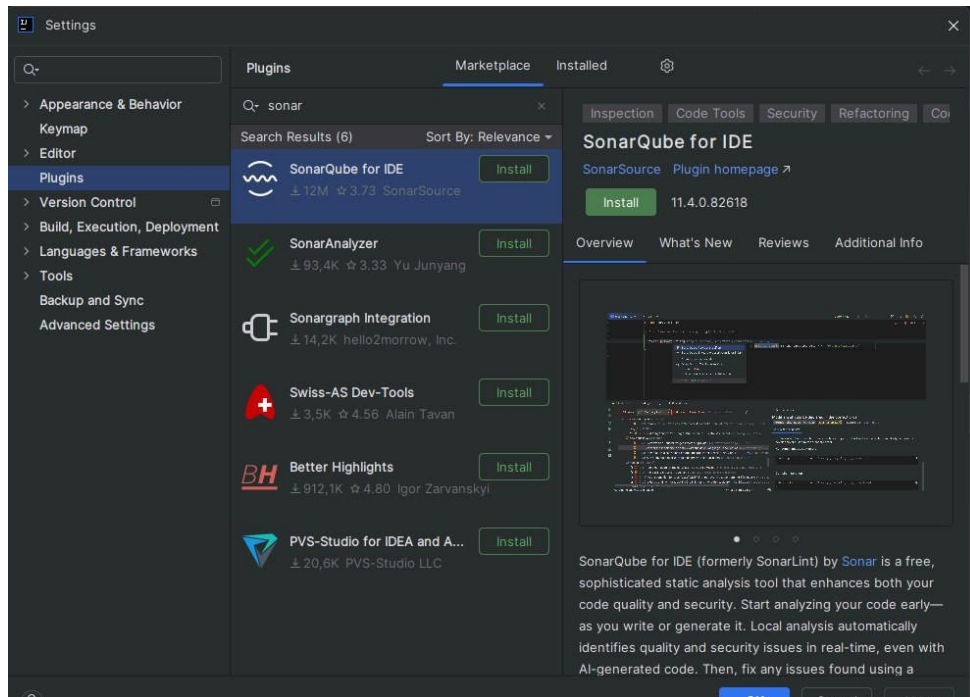
The file should look like this and must be placed in the root of your project. See the example below:



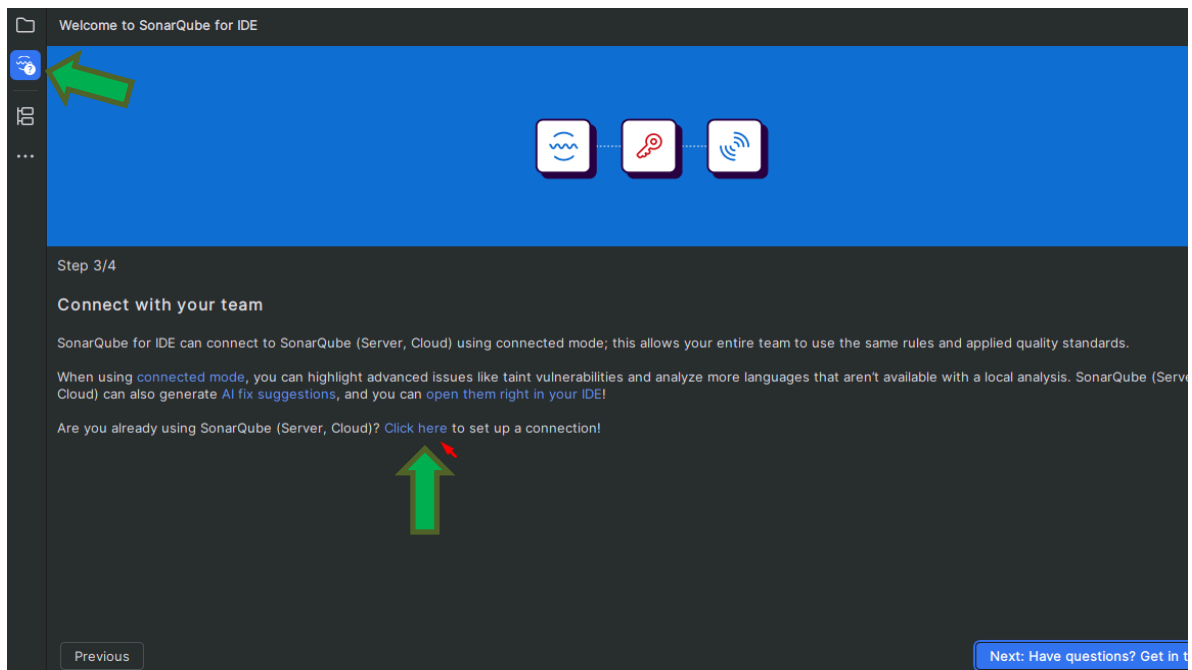
Later you'll have to create a "ci-settings.xml" file and put in the root of the project also, because the file above refers to this new file. You'll have an example of this new file at the end of the project.

2. Optional: SonarQube Cloud for IntelliJ

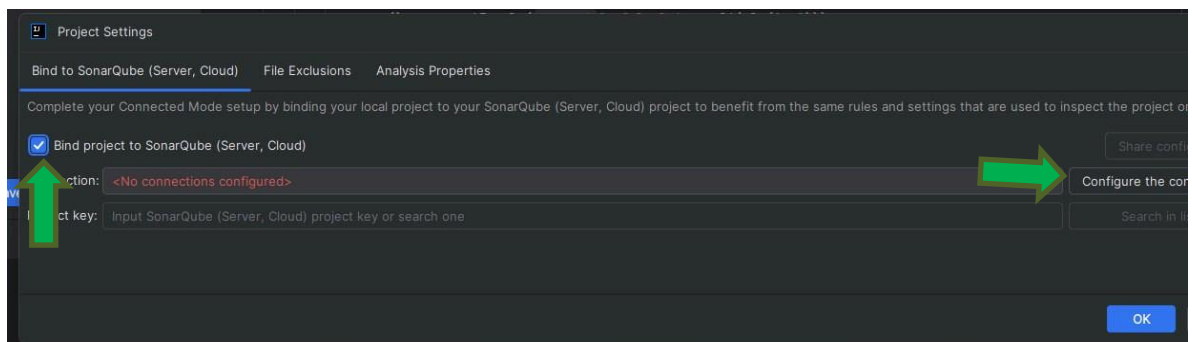
To have code quality metrics directly as you code within the IDE, download the plugin "SonarQube for IDE":



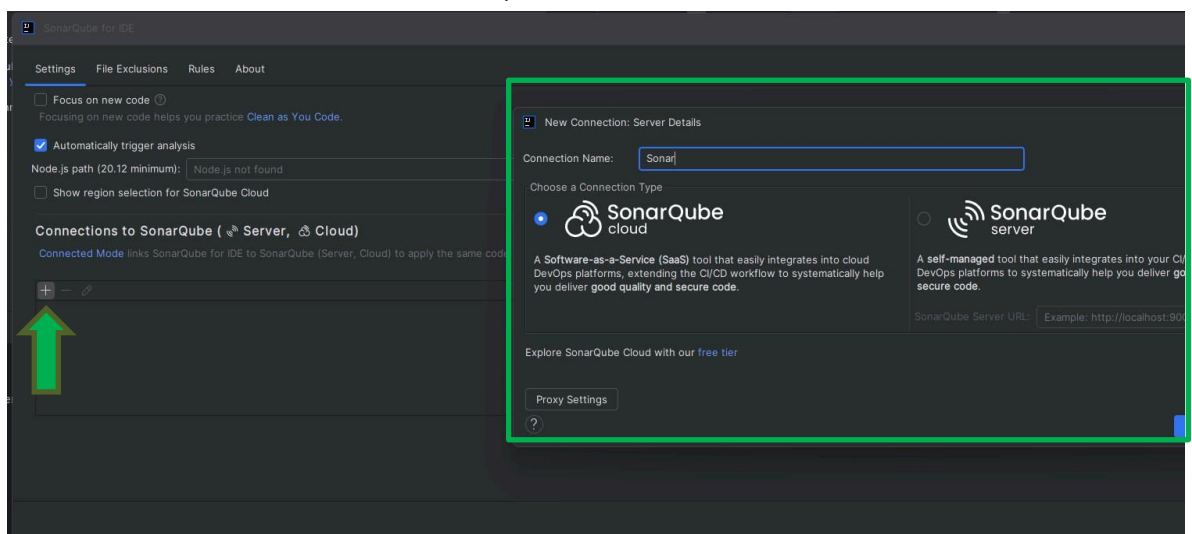
You'll see a new icon on your left bar. If you click on it, just choose next until you arrive to the 3rd step. Then click on "Click here".



In this window, just check the Bind box, and click "Configure the connection".

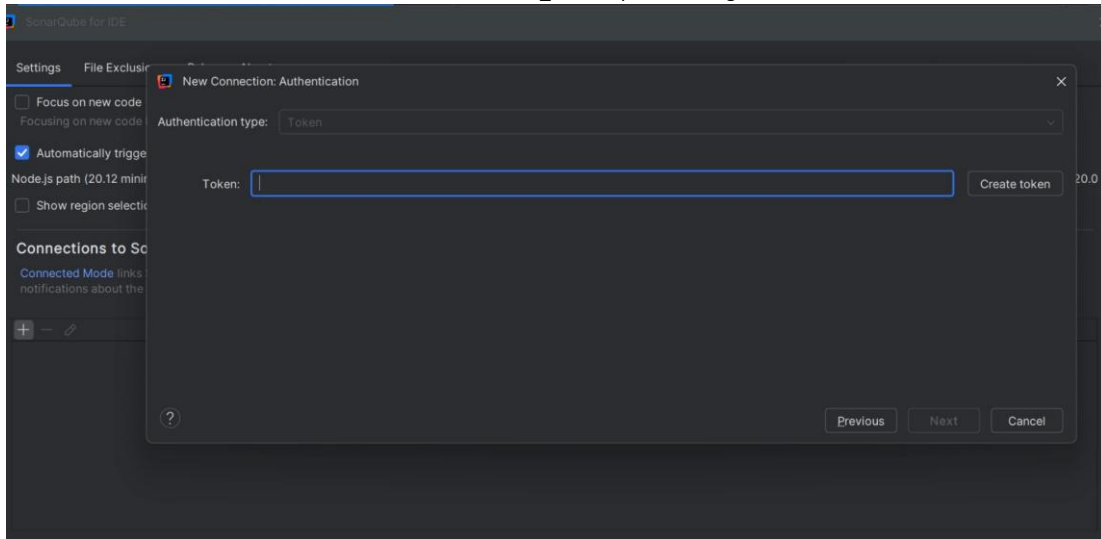


Click on the + button to add a new connection, and it will open a new modal window. Choose a name for the connection, choose SonarQube Cloud and click ok.

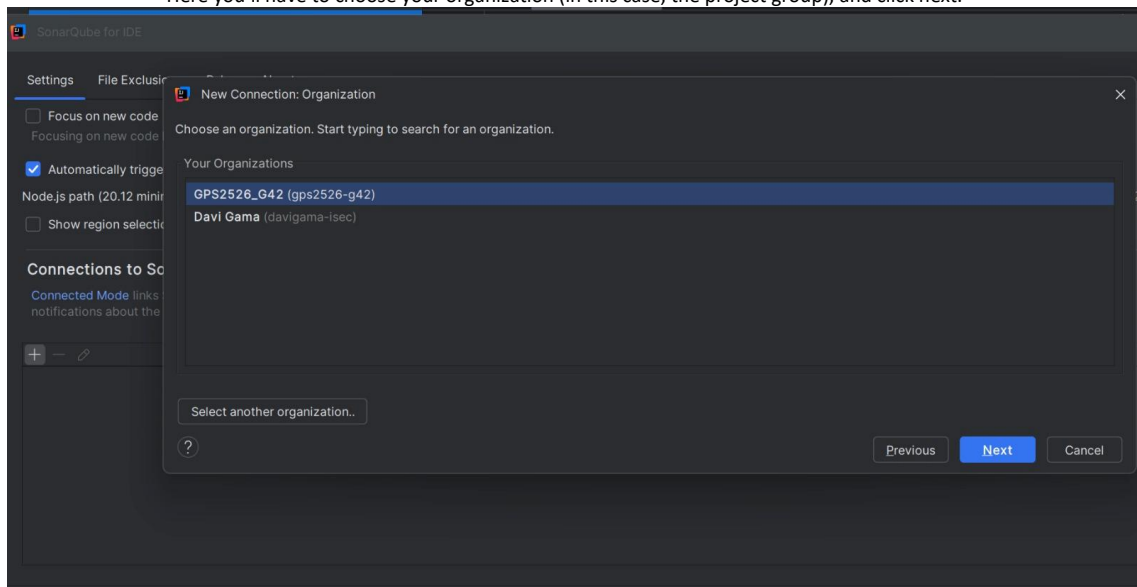


Walkthrough SonarQube Cloud by Rui Casaca and Davi Gama

The token used to create the first variable, SONAR_TOKEN, put it here again, and click next.

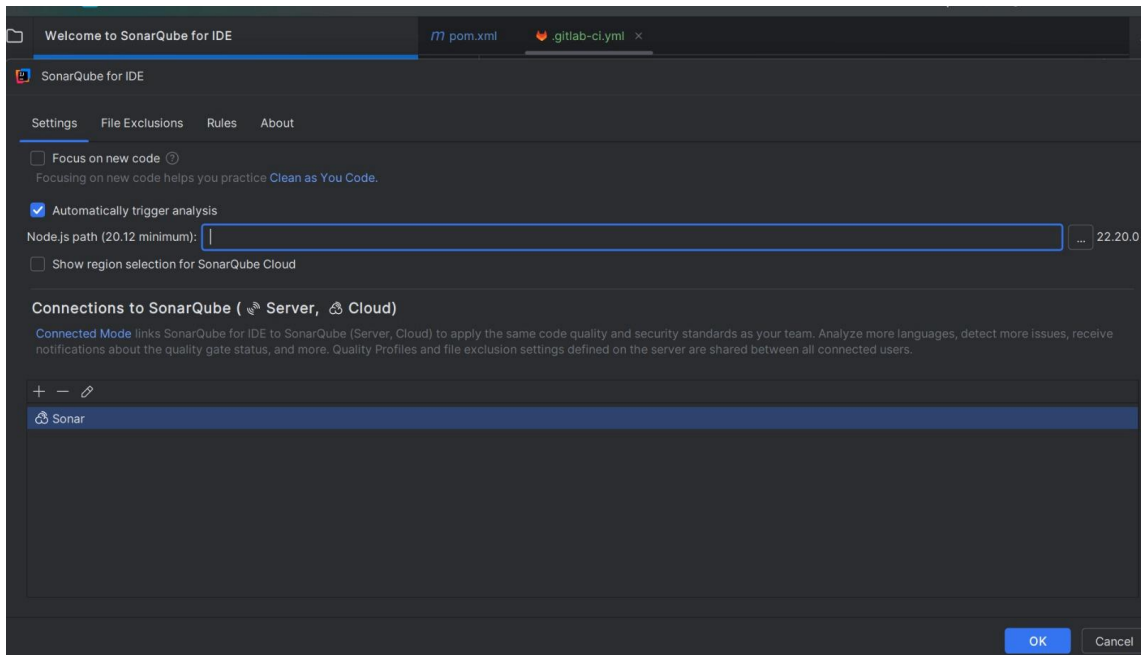


Here you'll have to choose your organization (in this case, the project group), and click next.



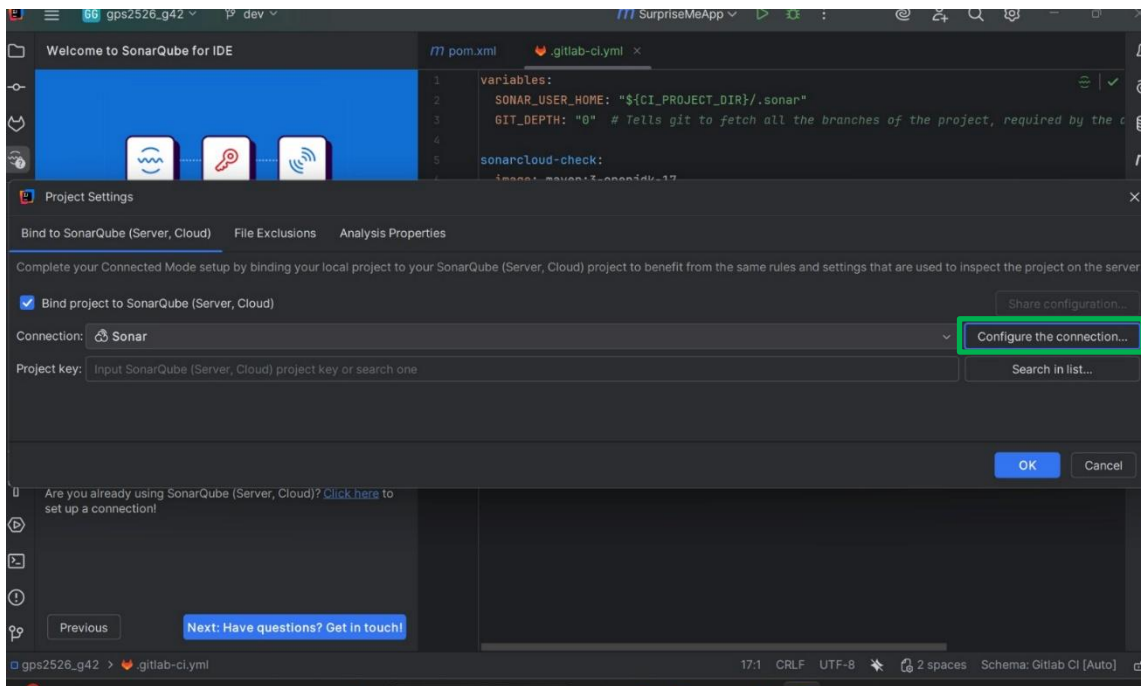
Walkthrough SonarQube Cloud by Rui Casaca and Davi Gama

Now you'll see that the new connection appears in the list. Just click ok.

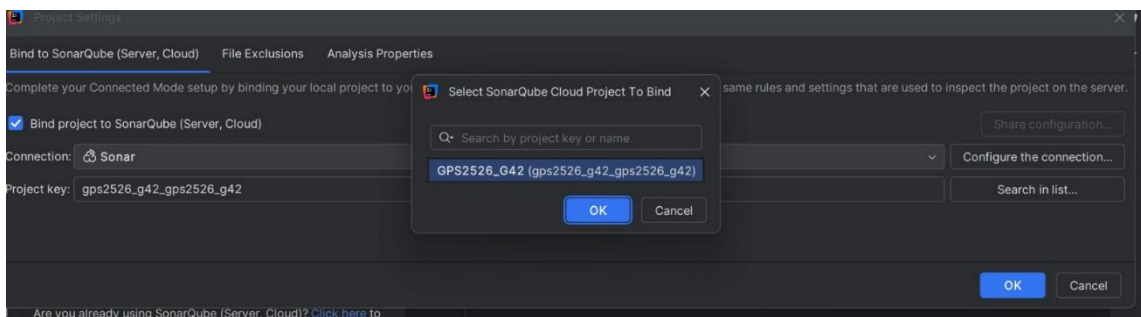


Walkthrough SonarQube Cloud by Rui Casaca and Davi Gama

Click on the “Configure the connection” button.

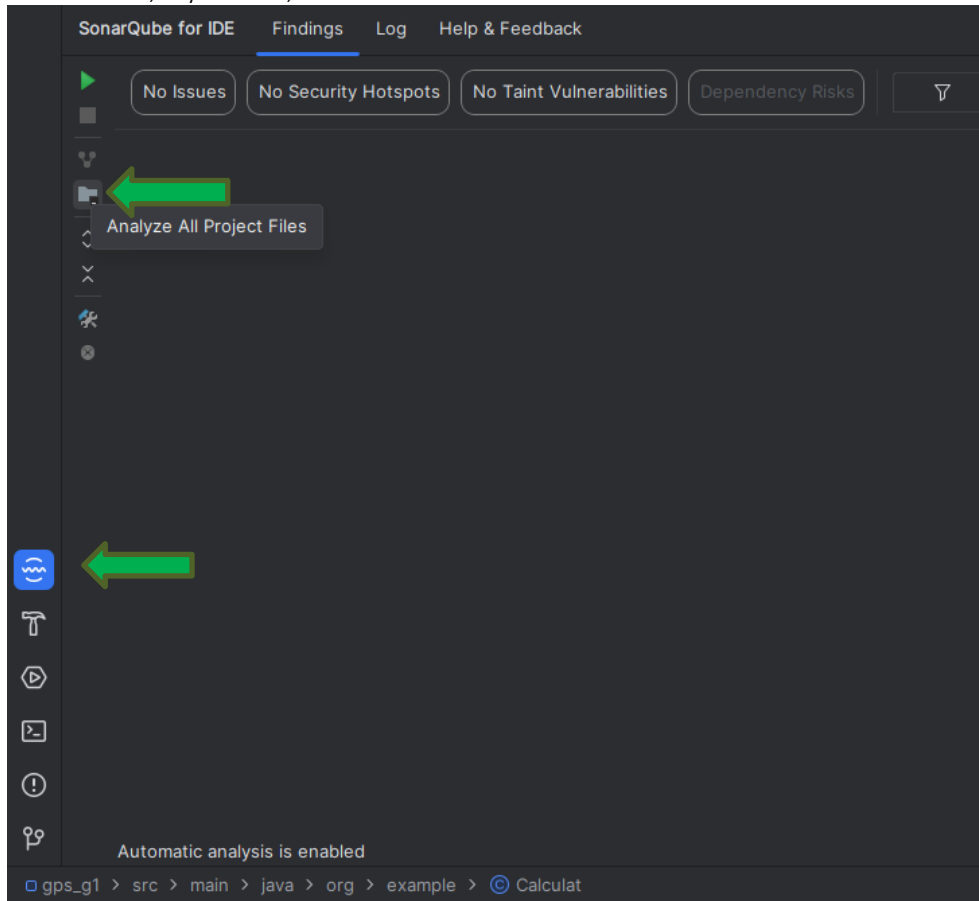


After opening the new connection, you’ve just created, select your project, and click ok on both buttons.

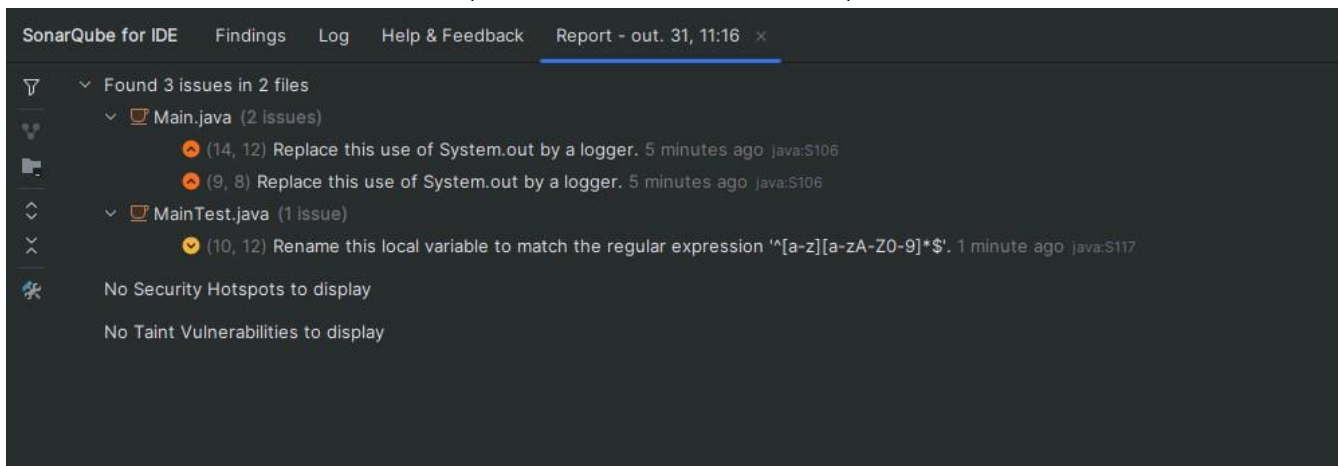


Walkthrough SonarQube Cloud by Rui Casaca and Davi Gama

Click on the icon, on you left bar, and on the folder icon.

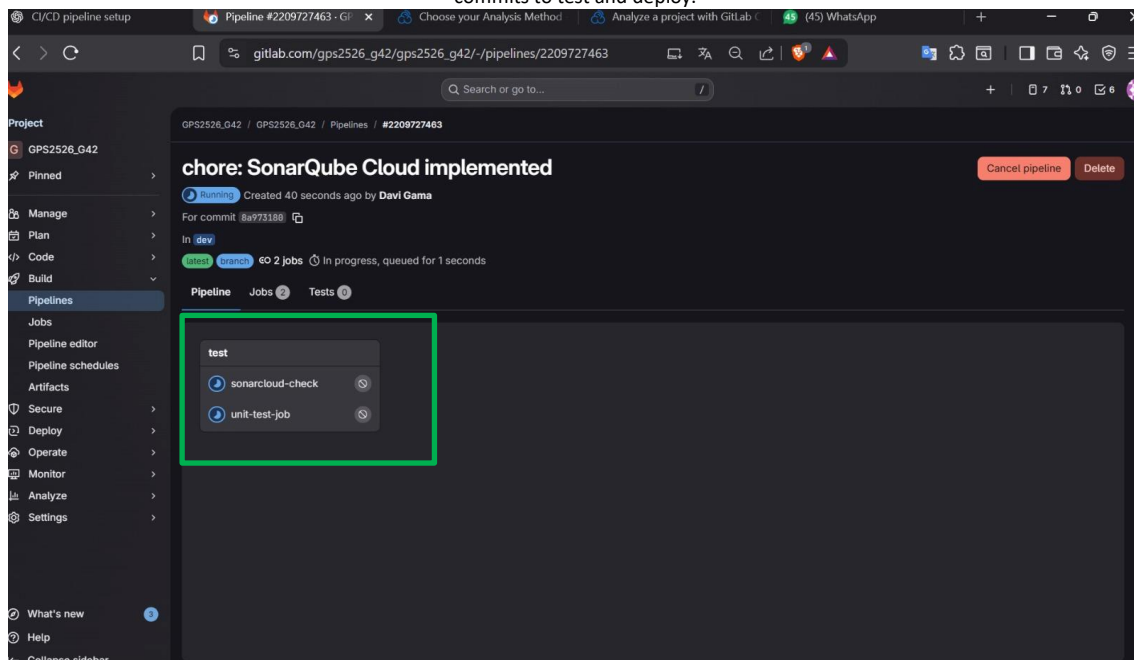


Then you should be able to see issues detected by Sonar.



Walkthrough SonarQube Cloud by Rui Casaca and Davi Gama

In the end, after you make git push command, in GitLab, you should see something like this appear and be ready to receive commits to test and deploy.



Expected Results

- SonarCloud configured.

Recommendations

- Install the plugin in your IDE to clean as you code.
- You'll need to make a git push command to submit these configurations.
- You'll also need to configure the ".gitlab-ci.yml" file to be able to configure and enable the pipelines process (CI/CD) in GitLab (See the next section).

Additional Resources

- [SonarQube Cloud documentation](#)

Example of the “.gitlab-ci.yml” file:

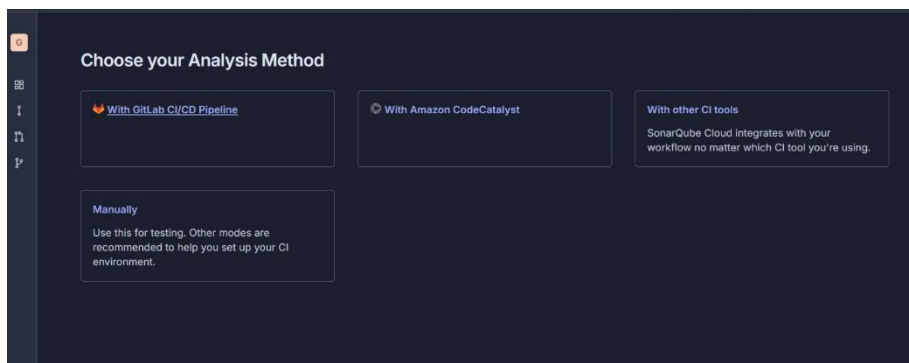
```
# GitLab CI/CD Pipeline Configuration
# This file defines the continuous integration and deployment pipeline for a Java Maven project.
# It includes building the application, running tests, and performing code quality checks.
# The pipeline is triggered on commits to the QA and main branches.

stages:
  - build
  - test
build:
  # Build job: Compiles, packages, and deploys the Java application using Maven
  stage: build
  image: maven:3.9.11
  script:
    - export NEW_VERSION=${CI_COMMIT_SHA:0:8}-${CI_COMMIT_REF_NAME}
    - mvn versions:set -DnewVersion=1-$NEW_VERSION
    - mkdir -p teste
    - mvn clean package deploy -s ci_settings.xml
  artifacts:
    paths:
      - target/*.jar
  only:
    - QA
    - main

unit-test-job:
  # Unit Test Job: Executes unit tests and generates JUnit reports
  image: maven:3.9.11
  stage: test
  script:
    - echo "Running unit tests..."
    - mvn test
  artifacts:
    when: always
    reports:
      junit: target/surefire-reports/*.xml

# Global variables for the pipeline
variables:
  SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar"
  GIT_DEPTH: "0"
sonarcloud-check:
  # SonarCloud Check: Performs static code analysis for quality and security
  image: maven:3.9.11
  cache:
    key: "${CI_JOB_NAME}"
    paths:
      - .sonar/cache
  script:
    - echo "Running SonarCloud analysis.."
    - mvn verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -Dsonar.projectKey=<PROJECT_KEY>
  stage: test
```

Aqui é necessário inserir o id do projeto. Esse id pode ser encontrado no URL do nosso projeto (o que vem a seguir ao “?id=” até ao fim), quando estamos nesta página (ver imagem abaixo). A key não precisa dos < >, basta colar o texto depois do “=”.



Example of the ci-settings.xml

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.1.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.1.0 http://maven.apache.org/xsd/settings-1.1.0.xsd">
  <servers>
    <server>
      <id>gitlab-maven</id>
      <configuration>
        <httpHeaders>
          <property>
            <name>Job-Token</name>
            <value>${CI_JOB_TOKEN}</value>
          </property>
        </httpHeaders>
      </configuration>
    </server>
  </servers>
</settings>
```